Running Review Doc for Math 128A

Ethan Ebbighausen

February 24, 2026

Contents

This document gives a running review of our Math 128A course, taught in Spring 2026 by Professor Ryan A. Hass using Kincaid and Cheney's book. This review is not comprehensive, and is just to give an overview of the topics.

1 Week 6: (Current) Gaussian Elimination (4.3)

2 Week 5: Computing roots of polynomials (3.5), Matrix Algebra and Factorizations (4.1-4.2)

2.1 Polynomial Roots

Due to the work of Gauss, we know that a degree n polynomial has n complex roots (the Fundamental Theorem of algebra paired with polynomial division). Following our trend on root finding to replace various proceeses like rational roots, we would like to find an efficient method of computing these roots. We start with localization theorems, which tell us a range of where the roots may lie. Consider a polynomial $p(x) = a_n z^n + a_{n-1} z^{n-1} + ... + a_0$.

Localization Theorem All the roots of $p(x)$ written as above lie in $B(0, \rho) \subset \mathbb{C}$ where $\rho = 1 + |a_n|^{-1} \max_{0 \leq k \leq n} |a_k|$.

Proof: Let $c = \max_{0 \leq k \leq n} |a_k|$. The case $c = 0$ is immediate, so assume $c > 0$ and $\rho > 1$. For $|z| \geq \rho$, (use the reverse triangle inequality and geometric sums).

$$|p(z)| \geq |a_n z^n| - c \sum_{k=1}^{n-1} |z|^k > |a_n z^n| - c|z|^n (|z| - 1)^{-1} \geq |a_n z^n|(1 - c|a_n|^{-1}(\rho - 1)^{-1}) = 0$$

QED

This theorem implies a second:

Corollary: If all the roots of $z^n p(1/z)$ are in the disk $B(0, \rho)$, then all the non-zero roots of p are outside $B(0, \rho^{-1})$.

Proof: The polynomial $z^n p(1/z)$ is $s(z) = a_n + a_{n-1} z + ... + a_0 z^n$. Then, $p(z_0) = 0$ is equivalent to $s(1/z_0) = 0$. Since the roots of s have $|z| \leq \rho$, we then have $\frac{1}{|z|} \geq \rho^{-1}$, as desired. *QED*

These results give a starting point for algorithms to take hold. We will focus on Newton's method, but the text also covers Bairstow's method and Laguerre iteration. For Newton's method, we want an easy way to comput the polynomial, especially centered at different points or in the nested multiplication form from the first week. For this, we use Horner's method.

Starting with a polynomial $p(z)$ of the above form, we perform an artificial polynomial division to reduce the degree of the polynomial. Since $p(z) - p(z_0)$ has a root at z_0, we may form $q(z) = \frac{p(z)-p(z_0)}{z-z_0}$ a degree $n-1$-polynomial such that $p(z) = p(z_0) + q(z)(z - z_0)$. Horner's method tells us how to compute this $q(z)$ and $p(z_0)$. Let $q(z) = b_{n-1}z^{n-1} + ... + b_0$. Then, using the product relationship $p(z) = p(z_0) + q(z)(z - z_0)$, $a_n = b_{n-1}$, and $a_j + z_0 b_j = b_{j-1}$ for $0 \leq j \leq n-1$. Rewriting this relationship, we have $b_k = a_{k+1} + z_0 a_{k+2}$. We may compute this all the way down to b_0, and if we put in $k = -1$, the value b_{-1} we solve for is actually $p(z_0)$.

One way this helps our algorithms on the computer side is that it allows us to compute $p'(z_0)$ without needed to use symbolic logic in the comuter (i.e. without using the power rule). Using a taylor series on p, we may see

$$p(z) = c_n(z - z_0)^n + ... + c_1(z - z_0) + c_0$$

where $c_1 = p'(z_0)$. Applying Horner's method at z_0 gives $p(z) = (z - z_0)q(z) + c_0$. It then follows that $q(z_0) = p'(z_0)$ from the product rule. Another way to see this is to write

$$p(z) = c_0 + (z - z_0)(c_1 + (z - z_0)(c_2 + ... + (z - z_0)c_n)...)$$

where, aligning with Horner's relationship, $q(z) = (c_1 + (z - z_0)(c_2 + ... + (z - z_0)c_n)...)$. Repeating Horner's method on q then gives the higher Taylor coefficients of p: for $p(z) = (z - z_0)q_0(z) + p(z_0)$ and $q_i(z) = q_{i+1}(z)(z - z_0) + q_i(z_0)$; then $q_k(z_0) = p^{(k)}(z_0)/k!$ and these values are the last value produced in each step of Horner's method.

We may now work on finding roots. In the special case z_0 is a root, our formula is simply $q(z) = \frac{p(z)}{z-z_0}$, so this division merely decreases the degree of p without any added complication. We may apply Newton's method to $p(z)$, computing its values and its derivative's values by Horner's method, then have $p(z) = (z - z_0)q(z)$ and repeat this idea on $q(z)$. In a finite number of steps, we will have fully factored $p(z)$.

The localization methods give us a good starting area, and we may use polynomial coefficients or estimates to get an idea on a radius that implies convergence for Newton's method. We also have the following:

Theorem on Successive Newton Estimates (Bodewig 1946) Let x_k and x_{k+1} be two successive iterates when NEwton's method is applied to a degree n polynomial p. Then, there is a root of p within distance $n|x_k - x_{k+1}|$ of x_k in the complex plane.

Proof Let $r_1, ..., r_n$ be the roots of p. Then, $p(z) = c\prod_{j=1}^{n}(z - r_j)$, and

$$p'(z) = c\sum_{i=1}^{n}\prod_{j\neq i}(z - r_j) = \sum_{i=1}^{n}\frac{p(z)}{z - r_i}$$

We claim that for and z, there is some index j so $|z - r_j| \leq n|p(z)/p'(z)|$. If this were to fail, we would have

$$|z - r_j|^{-1} < \frac{1}{n}|p'(z)/p(z)| = \frac{1}{n}|\sum(z - r_i)^{-1} \leq \frac{1}{n}\sum|z - r_i|^{-1}$$

Since the number on the right is an average, this is impossible. Notice now that $n|x_k - x_{k+1}| = n|-\frac{p(x_k)}{p'(x_k)}|$, so that we have shown root is within this radius of x_k.

QED

This theorem may be useful for your code or HW, and gives us an idea of how to set error bounds.

2.2 Matrix Algebra and Factorizations

In Math 54, we learned several factorizations like diagonalization, LU-factorization, QR-factorization, and Singular Value decomposition. These will be useful in making code and algorithms more efficient, so we review. Let $A \in M(n, \mathbb{R})$, which means A is a $n \times n$ matrix with entries in $\mathbb{R}$. We denote $A = (a_{i,j})$ to say

that the $a_{i,j}$ are the entries of A in the i, j positions. We also sometimes denote the rows of A by A_i or the columns by A_j.

Recall that we do many processes like Gaussian elimination, row reduction, and even diagonalization by elementary row operations. These operations include adding rows, interchanging rows, and multiplying rows by a nonzero constant. For each operation, there exists a matrix E so EA is the same as doing the row operation to A. These matrices E are invertible, and their inverse signifies the opposite row operation. Recall that EA and AE may be different, as multiplication is not commutative for matrices. You should review what being an invertible matrix means, how to compute simple inverses, and how invertibility for a square matrix is the same as being injective, surjective, being the product of elementary matrices, or having nonzero determinant. There are some guiding questions on a MAth 54 review doc from my older teaching sites.

We begin with LU-factorization. Recall that L is a lower triangular matrix if all entries strictly below the diagonal are 0, or $l_{i,j} = 0$ for $i < j$. We may similarly have an upper triangular matrix U. These types of matrices are useful for numerical methods because solving $Lx = b$ or $Ux = b$ is quite easy compared to for a standard matrix. For example, consider

$$\begin{bmatrix} a & 0 \\ b & c \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \end{bmatrix}$$

Then, we have $ax_1 = d_1$, so $x_1 = d_1/a_1$. This helps us solve the second equation, $bx_1 + cx_2 = d_2$ then becomes $x_2 = c^{-1}(d_2 - bd_1a_1^{-1})$. For larger matrices, the same recursion applies to simplyifying inverting L.

A square matrix A has an LU factorization if $A = LU$ where L is lower triangular and U is upper triangular. Like many other factorizations, this is not unique: if D is an invertible diagonal matrix, and $A = LU$, then $A = (LD)(D^{-1}U)$ is another LU-factorization. When A is invertible, it is possible to show that all LU factorizations of A are related by such a relationship.

What criteria do we have to imply an LU-factorization exists? Let $B \in M(k)$ be a sub-matrix of A. The leading principal matrices of A are those B where $b_{i,j} = a_{i,j}$ for all i, j, or where B is located in the top left corner of A. If all leading principal matrices of A are invertible, then A has n LU-factorization (the proof of this is semi-long, but should be understood. I will replicate it here at a later date).

We usually compute the LU-factorization by applying row operations to A to get an upper triangular matrix U, then collecting the elementary matrices to what should be a lower triangular matrix for L (if an LU factorization even exists). For example, consider the matrix

$$\begin{bmatrix} 8 & 4 \\ 4 & 1 \end{bmatrix}$$

By subtracting the half of the first row from the second, we obtain an upper triangular matrix

$$U = \begin{bmatrix} 8 & 4 \\ 0 & -1 \end{bmatrix}$$

and this elemetary row operation is encoded by $\begin{bmatrix} 1 & 0 \\ -0.5 & 1 \end{bmatrix}$. The inverse of this operation is $L = \begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix}$, and so we have $A = LU$ is an LU-factorization.

A more foolproof way to compute it theoeretically would be to set up systems of equations, but such a method includes exactly the type of problem we are trying to solve. Simplifying this system leads to the special case of Doolittle's factorization, an LU-factorization with $l_{i,i} = 1$ for $i = 1, ..., n$ (depending on your reference, these may just be nonzero, which is equivalent by the comments above about diagonal matrices). We may then recursively solve for the values of L and U using the equations

$$u_{k,j} = a_{k,j} - \sum_{s=1}^{k-1} l_{k,s} u_{s,j}$$

$$l_{i,k} = \left[a_{i,k} - \sum_{s=1}^{k-1} l_{i,s} u_{s,k} \right] / u_{k,k}$$

Try multiplying out abstract matrices in the 3×3 case to make sense of these equations. You should notice things like the first row of U and the first row of A being the same, and move from there.

Similarly, Crout factorization is an LU-factorization with $u_{i,i} = 1$ for all i, and also has a recursive algorithm for computing the entries of L and U. We also have a decomposition that looks to LU-factorization as orthogonal diagonalization does to diagonalization. Cholesky's factorization is an LU-factorization where $U = L^T$, or $A = LL^T$. Such a factorization exists and is unique if A is a symmetric and positive definite matrix, but may exist in much broader cases.

3 Week 4: Fixed Points and Functional Iteration (3.4)

The generalizations of Newton's method may be posed as a type of function iteration. If we define $F(x) = x - \frac{f(x)}{f'(x)}$, where f is the function we are trying to find a root of, then Newton's method becomes $F(x_n) = x_{n+1}$. This is a useful situation because there is a strong theorem in analysis that helps us find limits in such cases, called the contraction mapping theorem.

This theorem supposes that we have a non-empty complete metric space X, and defines a function $F : X \to X$ to be a contraction if $|F(x) - F(y)| \leq c|x - y|$ where $c \in (0, 1)$. Under this condition, it poses that we have a unique fixed point x^*, i.e. a point where $F(x^*) = x^*$, and the proof of the theorem shows that we may reach such x^* by taking any point y and $F^n(x) = F(F(...F(x)...))$ converges to x^* as $n \to \infty$.

In the context of the Secant method or Newton's method, we then converge to a root if we can pose appropriate F as a contraction. The MVT is often helpful in showing appropriate functions are contractions on the right spaces.

Finally, we care about the rate of convergence. Given F and $x_{n+1} = F(x_n)$, where F is a contraction with fixed point x^*, we trace $e_n = x_n - x^*$. Under suitable assumptions,

$$e_{n+1} = F(x_n) - F(s) = F(s + e_n) - F(s) = e_n F'(s) + \frac{1}{2} e_n^2 F''(s) + ... + \frac{1}{(q-1)!} e_n^{q-1} F^{(q-1)}(s) + \frac{1}{q!} e_n^q F^{(q)}(\zeta_n)$$

so that if s is an order -q root of F, then $e_{n+1} = \frac{1}{q!} e_n^q F^{(q)}(\zeta_n)$ and we have q-order convergence. Try checking the convergence for Newton's method and the Secant method.

Example: Determine the order of convergence of the sequence $x_n = n^{-1/2}$. We should expect linear convergence from normal limit arguments, but let us check using this method. Squaring the terms, we see $x_n^2 = n$, so that $x_n^{-2} + 1 = n + 1 = x_{n+1}^{-2}$. This is to say

$$F(x) = \frac{1}{\sqrt{\frac{1}{x^2} + 1}} = \frac{x}{\sqrt{1 + x^2}}$$

$$F'(x) = \frac{1 - x^2}{(x^2 + 1)^{3/2}}$$

so that $F'(0) \neq 0$. We expect linear convergence, and this may also be checked directly from the definition of the order of convergence.

3.1 Aitken's Extrapolation Formula

Let $F \in C^1(\mathbb{R})$ be a contraction with fixed point s, and $x_{n+1} = F(x_n)$ defining a sequence converging to that point. Then,

$$s - x_n \approx \frac{\lambda_n}{1-\lambda_n}(x_n - x_{n-1})$$
$$\lambda_n = \frac{x_n - x_{n-1}}{x_{n-1} - x_{n-2}}$$

Aitken's method stipulates that to compute s, starting with x_1, we compute $x_2 = F(x_1)$ and $x_3 = F(x_2)$, then compute x_4 via the above formula. To proceed, we compute $x_n = F(x_{n-1})$ if $n \equiv 1, 2, 3 \mod 4$ and use Aitken's formula for x_{4k}.

4 Week 3: 3 Root finding Algorithms (3.1-3.3)

4.1 Bisection Method

The bisection method starts with a *continuous* function f and an interval $[a, b]$ on which we have $f(a)f(b) < 0$. This is a special assumption to guarantee that we have a root of our function f on $[a, b]$ by the Intermediate Value Theorem.

We then produce $c = \frac{a+b}{2} = a + \frac{b-a}{2}$, the midpoint of the interval. We check whether $f(a)f(c) < 0$, in which case we set $[a_1, b_1] = [a, c]$. In the case $f(a)f(c) > 0$, we know $f(a)f(b) < 0$, so we et $[a_1, b_1] = [c, b]$. Finally, if $f(c) = 0$, we terminate. We repeat this process, producing c_1 and then $[a_2, b_2]$.

This way, each step cuts the interval in half. We use c_n as out approximation of the root. If r is the root we eventually converge to, we know $|c_n - r| \leq \frac{1}{2}|b_n - a_n| \leq 2^{-n}|b - a|$.

4.2 Newton's Method

For Newton's method, we assume that we have a C^2 function f for which we want to find a root. We start with some initial value x_0. Then, we assume there exists some root r, and so we know $0 = f(r) = f(x_0 - e_0)$ where $e_0 = x_0 - r$. Expanding by Taylor's theorem, $0 = f(x_0) - e_n f'(x_0) + ...$, so we approximate $e_0 \approx f(x_0)/f'(x_0)$ and update

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

at each subsequent step.

This isn't guaranteed to converge for any starting point, but let's look at the case of convergence.

$$e_{n+1} = x_{n+1} - r = x_n - \frac{f(x_n)}{f'(x_n)} - r = e_n - \frac{f(x_n)}{f'(x_n)}$$

Thus, we use that $0 = f(x_n) - e_n f'(x_n) + e_n^2 f''(\zeta_n)/2$ from above to rewrite this as

$$e_{n+1} = \frac{e_n f'(x_n) - f(x_n)}{f'(x_n)} = \frac{1}{2}\frac{f''(\zeta_n)}{f'(x_n)}e_n^2$$

When do we see convergence? We need the $e_n \to 0$ for this, which is okay when this coefficient is bounded. This is a second place where we use the C^2 assumption. Set

$$c(\delta) = \frac{1}{2}\frac{\max_{|x-r|<\delta}|f''(x|)}{\min_{|x-r|<\delta}|f'(x)|}$$

Using continuity, take δ small enough so $\delta c(\delta) < 1$. Then, it is easy to note that if $|e_n| \leq \delta$, then $|e_{n+1}| \leq \delta$ as well and that $|e_{n+1}| \leq C|e_n|^2$, so that we have quadratic convergence when x_0 is chosen within a δ-ball of r.

4.3 Alternatives to Newton's method

-Secant Method, Steffenson's method

5 Week 2: Floating Point Numbers and Machine Numbers (2)

-Add all

6 Week 1: Calculus Review (1)

- Add IVT, MVT, Taylor's Theorem